

SEQUENCES, CONTAINERS, OBJECTS, & LINKED LISTS Solutions

COMPUTER SCIENCE MENTORS 61A

September 28, 2026 – October 2, 2026

1 Sequences

Sequence Overview

Sequences are ordered data structures that have length and support element selection. Here are some common types of sequences you'll be dealing with in this class:

- Lists: `[1, [2], 'a', lambda x: 5]`
- Tuples: `(1, (2,), 'a', lambda x: 5)`
- Strings: `'Hello World!'`

While each type of sequence is different, they all share a common interface for manipulating and accessing their data:

- **Item selection:** Use square brackets to select an element at an index:

```
(3, 1, 2)[0] # returns 3
"Hello"[-1]  # returns "o"
```

- **Length:** The built-in `len` function returns the length of a sequence:

```
len((1, 2)) # returns 2
```

- **Concatenation:** Sequences can be concatenated with the `+` operator, which returns a *new* sequence:

```
[1, 2] + [3, 4] # returns [1, 2, 3, 4]
```

- **Membership:** The `in` operator tests for sequence membership:

```
1 in (1, 2, 3)          # returns True
5 not in (1, 2, 3)      # returns True
"apple" in "snapple"    # returns True
```

Membership in Strings vs. Lists and Tuples: As a short aside, while the `in` operator works the same for lists and tuples, checking if an element is contained within the list/tuple container, the `in` operator instead for strings checks for direct substrings rather than the existence of distinct elements within the string.

- **Looping:** Sequences can be looped through with `for` loops:

```
>>> for x in [1, 2, 3]:
...     print(x)
1
2
3
```

- **Aggregation:** Common built-in functions—including **sum**, **min**, and **max**—can take sequences and aggregate them into a single value:

```
max((3, 4, 5)) # returns 5
```

- **Slicing:** Slicing is a way to create a copy of all or part of a sequence. The general syntax for slicing a sequence `seq` is as follows:

```
seq[<start index>:<end index>:<step size>]
```

This evaluates to a new sequence that includes every element starting at `<start index>` and up to and *excluding* `<end index>` in `seq`, taking steps of size `<step size>`.

If we do not supply `<start index>` or `<end index>`, it will start at the beginning of the sequence and include every element up to and including the end of the sequence.

```
>>> lst = [1, 2, 3, 4, 5]
>>> lst[2:]
[3, 4, 5]
>>> lst[:3]
[1, 2, 3]
>>> lst[::-1]
[5, 4, 3, 2, 1]
>>> lst[1:2]
[2, 4]
```

List comprehensions create a new list by processing an iterable, such as a list, tuple, string, or range. The syntax for a list comprehension is

```
[<expression> for <element> in <sequence> if <condition>]
```

We could equivalently write the following:

```
lst = []
for <element> in <sequence>:
    if <condition>:
        lst = lst + [<expression>]
```

The **if** `<condition>` filter statement is optional. The following list comprehension doubles each odd element of `[1, 2, 3, 4]`:

```
>>> [i * 2 for i in [1, 2, 3, 4] if i % 2 != 0]
[2, 6]
```

Equivalent in **for** loop syntax:

```
lst = []
for i in [1, 2, 3, 4]:
    if i % 2 != 0:
        lst = lst + [i * 2]
```

Sequence Practice

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]
>>> a
```

```
[1, 2, 3]
```

```
>>> a[2]
```

```
3
```

```
>>> a[-1]
```

```
3
```

```
>>> b = a
>>> a = a + [4, [5, 6]]
>>> a
```

```
[1, 2, 3, 4, [5, 6]]
```

```
>>> b
```

```
[1, 2, 3]
```

```
>>> c = a
>>> a = [4, 5]
>>> a
```

```
[4, 5]
```

```
>>> c
```

```
[1, 2, 3, 4, [5, 6]]
```

```
>>> d = c[3:5]
>>> c[3] = 9
>>> d
```

```
[4, [5, 6]]
```

```
>>> c[4][0] = 7
>>> d
```

```
[4, [7, 6]]
```

```
>>> c[4] = 10
>>> d
```

```
[4, [7, 6]]
```

```
>>> c
```

```
[1, 2, 3, 9, 10]
```

2. Write a function `duplicate_list`, which takes in a list of positive integers and returns a new list with each element `x` in the original list duplicated `x` times.

```
def duplicate_list(lst):
    """
    >>> duplicate_list([1, 2, 3])
    [1, 2, 2, 3, 3, 3]
    >>> duplicate_list([5])
    [5, 5, 5, 5, 5]
    """
    _____

    for _____:
        for _____:
            _____
        _____

    new_list = []
    for x in lst:
        for i in range(x):
            new_list = new_list + [x]
    return new_list
```

3. Write a function that takes in a list `nums` and returns a new list with only the primes from `nums`. Assume that `is_prime(n)` is defined. You may use a **while** loop, a **for** loop, or a list comprehension.

```
def all_primes(nums):
```

```
    result = []
    for i in nums:
        if is_prime(i):
            result = result + [i]
    return result
```

```
    List comprehension:
```

```
    return [x for x in nums if is_prime(x)]
```

4. Fill in the methods below according to the doctests.

```
def gen_list(n):
    """
    Returns a nested list structure of n elements where the
    ith element is a list from 0 (inclusive) to i (exclusive).
    >>> gen_list(3)
    [[0], [0, 1], [0, 1, 2]]
    >>> gen_list(5)
    [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]
    """
    return _____
```

For an additional challenge, try out the following:

```
def gen_increasing(n):
    """
    Returns a nested list structure of n elements where the
    ith element of each list is one more than the previous
    element (even if the previous is in a prior sublist).
    >>> gen_increasing(3)
    [[0], [1, 2], [3, 4, 5]]
    >>> gen_increasing(5)
    [[0], [1, 2], [3, 4, 5], [6, 7, 8, 9], [10, 11, 12, 13,
    14]]
    """
    return _____
```

Hint: You can sum ranges. E.g. `sum(range(3))` gives us $0 + 1 + 2 = 3$.

```
def gen_list(n):
    return [[i for i in range(j+1)] for j in range(n)]

def gen_increasing(n):
    return [[i for i in range(sum(range(j+1)), sum(range(j+1)) + j+1)] for
            j in range(n)]
```

An alternate solution for `gen_increasing` is shown below:

```
def gen_increasing(n):
    return [[i + sum(range(j + 1)) for i in range(j + 1)] for j in
            range(n)]
```

2 Containers and Dictionaries

5. Write a function `count_t`, which takes in a dictionary `d` and a string `word`. The function should count the instances of the letter "t" in `word` and add a key-value pair to the dictionary. The key will be `word` and the value will be the number of "t"s in `word`

```
def count_t(d, word):  
    """  
    >>> words = {}  
    >>> count_t(words, "tatter")  
    >>> words["tatter"]  
    3  
    >>> count_t(words, "tree")  
    >>> words  
    {'tatter': 3, 'tree': 1}  
    """
```

```
    _____  
  
    for _____:  
        if _____:
```

```
    _____  
  
    count = 0  
    for c in word:  
        if c == 't':  
            count += 1  
    d[word] = count
```

6. Complete the function `snapshot`, which takes a single-argument function `f` and a list `snap_inputs` and returns a “snapshot” of `f` on `snap_inputs`. A “snapshot” is a dictionary where the keys are the provided `snap_inputs` and the values are the corresponding outputs of `f` on each input.

```
def snapshot(f, snap_inputs):  
    """  
    >>> snapshot(lambda x: x**2, [1, 2, 3])  
    {1: 1, 2: 4, 3: 9}  
    """  
  
    snap = _____  
  
    _____:  
        _____  
  
    return snap
```

```
def snapshot(f, snap_inputs):  
    snap = {}  
    for snap_input in snap_inputs:  
        snap[snap_input] = f(snap_input)  
    return snap
```

3 Objects and Data Abstraction

7. (a) What is an **Abstract Data Type**?

A structure that stores data but separates how it is stored from how it is used.

There are two different layers to abstract data types:

1) The program layer, which uses the data, and 2) The concrete data representation that is independent of the programs that use the data. The only communication between the two layers is through selectors and constructors that implement the abstract data in terms of the concrete representation.

In addition to selectors and constructors, there are behavior conditions under which the selectors and constructors give an appropriate response. That is, if we construct a rational number data type x from integers n and d , then it should be the case that $\text{numer}(x) / \text{denom}(x)$ is equal to n / d .

In general, we can think of an abstract data type as defined by some collection of selectors and constructors, together with some behavior conditions. As long as the behavior conditions are met (such as the division property above), these functions constitute a valid representation of the data type.

(b) What is the relationship between a class and an ADT?

Classes are a way to implement an Abstract Data Type. But, ADTs can also be created using a collection of functions, as shown by the rational number example.

8. What would Python display?

```
class Foo(object):  
    x = 'bam'  
  
    def __init__(self, x):  
        self.x = x  
  
    def baz(self):  
        return type(self).x + self.x
```

```
class Bar(Foo):  
    x = 'boom'  
  
    def __init__(self, x):  
        Foo.__init__(self, 'er' + x)
```

```
foo = Foo('boo')
```

(a) >>> Foo.x

'bam'

(b) >>> foo.x

'boo'

(c) >>> foo.baz()

'bamboo'

(d) >>> Foo.baz()

Error

(e) >>> Foo.baz(foo)

'bamboo'

(f) >>> bar = Bar('ang')

>>> Bar.x

'boom'

(g) >>> bar.x

'erang'

(h) >>> bar.baz()

'boomerang'

For each of the following problems, assume linked lists are defined as follows:

```
class Link:
    empty = ()
    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is not Link.empty:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '<'
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + '>'
```

9. What will Python output? Draw box-and-pointer diagrams along the way.

```
>>> a = Link(1, Link(2, Link(3)))
```

```
+---+---+ +---+---+ +---+---+
| 1 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
1
```

```
>>> a.first = 5
```

```
+---+---+ +---+---+ +---+---+
| 5 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
5
```

```
>>> a.rest.first
```

```
2
```

```
>>> a.rest.rest.rest.rest.first
```

```
Error: tuple object has no attribute rest (Link.empty has no rest)
```

```
>>> a.rest.rest.rest = a
```

```
      +---+---+ +---+---+ +---+---+
+-->| 5 | -->| 2 | -->| 3 | -->|
| +---+---+ +---+---+ +---+---+ |
|                                     |
+-----+-----+
```

```
>>> a.rest.rest.rest.rest.first
```

```
2
```

```
>>> repr(Link(1, Link(2, Link(3, Link.empty))))
```

```
"Link(1, Link(2, Link(3)))"
```

```
>>> Link(1, Link(2, Link(3, Link.empty)))
```

```
Link(1, Link(2, Link(3)))
```

```
>>> str(Link(1, Link(2, Link(3))))
```

```
'<1 2 3>'
```

```
>>> print(Link(Link(1), Link(2, Link(3))))
```

```
<<1> 2 3>
```

10. Write a function `linkify` that, given a list `lst`, returns a linked list with the same values. You may assume that there will be no nested lists.

Try writing this function both recursively and iteratively.

```
def linkify_rec(lst):
    """
    >>> lst = [0, 1, 2, 3]
    >>> linkify_rec(lst)
    Link(0, Link(1, Link(2, Link(3))))
    """
    if _____:
        _____

    else:
        _____

def linkify_iter(lst):
    _____

    for _____:
        _____

    _____

def linkify_rec(lst):
    if not lst:
        return Link.empty
    else:
        return Link(lst[0], linkify_rec(lst[1:]))

def linkify_iter(lst):
    retVal = Link.empty
    for elem in lst[::-1]:
        retVal = Link(elem, retVal)
    return retVal
```

11. Write a function `combine_two`, which takes in a linked list of integers `lnk` and a two-argument function `fn`. It returns a new linked list where every two elements of `lnk` have been combined using `fn`.

```
def combine_two(lnk, fn):
    """
    >>> lnk1 = Link(1, Link(2, Link(3, Link(4))))
    >>> combine_two(lnk1, add)
    Link(3, Link(7))
    >>> lnk2 = Link(2, Link(4, Link(6)))
    >>> combine_two(lnk2, mul)
    Link(8, Link(6))
    """
    if _____:
        return _____
    elif _____:
        return _____
    combined = _____
    return _____

def combine_two(lnk, fn):
    if lnk is Link.empty:
        return Link.empty
    elif lnk.rest is Link.empty:
        return Link(lnk.first)
    combined = fn(lnk.first, lnk.rest.first)
    return Link(combined, combine_two(lnk.rest.rest, fn))
```

5 Optional Extension: Tree-Recursive Counting

12. Gabe's Donut shop has an unlimited supply of f different flavors of donuts. Adit wants to buy a box containing d donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his d donuts from the f flavors. You may assume that d and f are non-negative integers.

Hint: Does order matter?

```
def donut(d, f):
    """
    >>> donut(12, 1)
    1
    >>> donut(12, 2)
    13
    >>> donut(12, 12)
    1352078
    >>> donut(0, 0)
    1
    """
    if _____:

        return _____

    if _____:

        return _____

    return _____

def donut(d, f):
    if d == 0:
        return 1
    if f == 0:
        return 0
    return donut(d - 1, f) + donut(d, f - 1)
```

Suppose the flavors are numbered 1 through f . Because order does not matter, we're going to say that Adit fills up his donut box starting with flavor number f and working down to flavor number 1. Under this scheme, Adit has a choice to make: does he want a donut of flavor number f , or does he not want one? If he does initially take a donut of flavor number f , then the number of ways to select the remaining $d - 1$ slots of the box is `donut(d - 1, f)`. If he does not want a donut of flavor number f , then he has d slots left in the box and $f - 1$ flavors to fill them with, making the number of ways to make the selection `donut(d, f - 1)`. Since he must make the choice one way or the other, the total number of ways to fill up the box is `donut(d - 1, f) + donut(d, f - 1)`.